


```

    if not pk:
        log.error("❌ PRIVATE_KEY not set!")
        return None
    try:
        return Keypair.from_bytes(b58decode(pk))
    except Exception as e:
        log.error(f"❌ Invalid private key: {e}")
        return None

keypair = load_keypair()

# — HELIUS HELPERS —————
async def get_wallet_transactions(session, wallet, limit=5):
    """Get recent transactions for a wallet."""
    url = f"https://api.helius.xyz/v0/addresses/{wallet}/transactions?api-key={HE
    try:
        async with session.get(url, timeout=aiohttp.ClientTimeout(total=10)) as r:
            if r.status == 200:
                return await r.json()
    except Exception as e:
        log.error(f"Error fetching txs for {wallet[:8]}...: {e}")
    return []

async def get_token_price(session, mint):
    """Get token price in SOL via Jupiter."""
    try:
        url = f"https://lite-api.jup.ag/price/v2?ids={mint}&vsToken={SOL_MINT}"
        async with session.get(url, timeout=aiohttp.ClientTimeout(total=5)) as r:
            if r.status == 200:
                data = await r.json()
                return float(data.get("data", {}).get(mint, {}).get("price", 0))
    except:
        pass
    return 0

# — JUPITER SWAP —————
async def swap(session, input_mint, output_mint, amount_lamports):
    """Execute a swap via Jupiter."""
    if not keypair:
        log.error("No keypair loaded, cannot swap!")
        return False

    try:
        # Get quote
        quote_url = (
            f"{JUPITER_URL}/quote?"
            f"inputMint={input_mint}&outputMint={output_mint}"

```

```

        f"&amount={amount_lamports}&slippageBps={SLIPPAGE_BPS}"
    )
    async with session.get(quote_url, timeout=aiohttp.ClientTimeout(total=10))
        if r.status != 200:
            log.error(f"Quote failed: {r.status}")
            return False
        quote = await r.json()

    # Get swap transaction
    swap_body = {
        "quoteResponse": quote,
        "userPublicKey": str(keypair.pubkey()),
        "wrapAndUnwrapSol": True,
        "dynamicComputeUnitLimit": True,
        "prioritizationFeeLamports": 100000,
    }
    async with session.post(
        f"{JUPITER_URL}/swap",
        json=swap_body,
        timeout=aiohttp.ClientTimeout(total=15)
    ) as r:
        if r.status != 200:
            log.error(f"Swap failed: {r.status}")
            return False
        swap_data = await r.json()

    # Sign and send
    import base64
    from solders.transaction import VersionedTransaction
    from solana.rpc.async_api import AsyncClient

    raw_tx = base64.b64decode(swap_data["swapTransaction"])
    tx = VersionedTransaction.from_bytes(raw_tx)
    signed = keypair.sign_message(bytes(tx.message))

    async with AsyncClient(HELIUS_RPC) as client:
        result = await client.send_raw_transaction(bytes(tx))
        sig = str(result.value)
        log.info(f"✅ Swap sent! Sig: {sig[:16]}...")
        return sig

except Exception as e:
    log.error(f"Swap error: {e}")
    return False

# ——— TRADE LOGIC ———
async def buy_token(session, token_mint, reason=""):

```

```

"""Buy a token with configured SOL amount."""
if token_mint in active_positions:
    log.info(f"Already holding {token_mint[:8]}..., skipping")
    return

amount_lamports = int(TRADE_SOL * 1_000_000_000)
log.info(f"🟢 BUYING {token_mint[:8]}... | {TRADE_SOL} SOL | {reason}")

sig = await swap(session, SOL_MINT, token_mint, amount_lamports)
if sig:
    price = await get_token_price(session, token_mint)
    active_positions[token_mint] = {
        "entry_price": price,
        "sol_spent": TRADE_SOL,
        "sig": sig,
        "time": datetime.now()
    }
    log.info(f"🇺🇸 Position opened: {token_mint[:8]}... @ {price:.10f} SOL")

async def sell_token(session, token_mint, reason=""):
    """Sell entire position of a token."""
    if token_mint not in active_positions:
        return

    log.info(f"🔴 SELLING {token_mint[:8]}... | {reason}")
    # Get token balance
    try:
        async with session.post(HELIUS_RPC, json={
            "jsonrpc": "2.0", "id": 1,
            "method": "getTokenAccountsByOwner",
            "params": [
                str(keypair.pubkey()),
                {"mint": token_mint},
                {"encoding": "jsonParsed"}
            ]
        }) as r:
            data = await r.json()
            accounts = data.get("result", {}).get("value", [])
            if not accounts:
                log.warning("No token balance found")
                return
            balance = int(accounts[0]["account"]["data"]["parsed"]["info"]["token

    if balance > 0:
        sig = await swap(session, token_mint, SOL_MINT, balance)
        if sig:
            del active_positions[token_mint]

```

```

        log.info(f"✅ Position closed: {token_mint[:8]}...")
    except Exception as e:
        log.error(f"Sell error: {e}")

# — TRANSACTION PARSER —
def parse_transaction(tx):
    """Extract transfers and swaps from a Helius enhanced transaction."""
    transfers_out = []
    token_buys = []

    tx_type = tx.get("type", "")
    source = tx.get("feePayer", "")

    # Native SOL transfers
    for transfer in tx.get("nativeTransfers", []):
        if transfer.get("fromUserAccount") == source:
            to = transfer.get("toUserAccount", "")
            amount = transfer.get("amount", 0) / 1e9
            if to and amount > 0.01: # Filter dust
                transfers_out.append({"to": to, "sol": amount})

    # Token swaps (buys)
    if tx_type in ("SWAP", "SWAP_EXACT_IN"):
        for swap_info in tx.get("events", {}).get("swap", []):
            token_out = swap_info.get("tokenOutputs", [{}])[0]
            mint = token_out.get("mint", "")
            if mint and mint != SOL_MINT:
                token_buys.append(mint)

    # Also check token transfers
    for t in tx.get("tokenTransfers", []):
        if t.get("toUserAccount") == source and t.get("mint") != SOL_MINT:
            token_buys.append(t.get("mint"))

    return transfers_out, token_buys

# — WALLET WATCHER —
async def watch_new_wallet(session, wallet, timeout=FOLLOW_TIMEOUT):
    """Watch a new wallet for trades after receiving a transfer."""
    log.info(f"👁️ Watching new wallet: {wallet[:8]}... for {timeout}s")
    deadline = asyncio.get_event_loop().time() + timeout
    seen_txs = set()

    while asyncio.get_event_loop().time() < deadline:
        txs = await get_wallet_transactions(session, wallet, limit=3)
        for tx in txs:
            sig = tx.get("signature", "")

```

```

        if sig in seen_txs:
            continue
        seen_txs.add(sig)

    _, token_buys = parse_transaction(tx)
    for mint in token_buys:
        log.info(f"🎯 Followed wallet {wallet[:8]}... bought {mint[:8]}!")
        await buy_token(session, mint, reason=f"Follow {wallet[:8]}...")

    await asyncio.sleep(3)

log.info(f"🕒 Done watching {wallet[:8]}...")
followed_wallets.pop(wallet, None)

# ——— MAIN MONITOR ———
async def monitor_target(session):
    """Monitor the target wallet via WebSocket."""
    log.info(f"🚀 Starting wallet monitor for {TARGET_WALLET[:8]}...")
    seen_txs = set()

    async with aiohttp.ClientSession() as ws_session:
        while True:
            try:
                async with ws_session.ws_connect(HELIUS_WS, heartbeat=30) as ws:
                    # Subscribe to target wallet
                    await ws.send_json({
                        "jsonrpc": "2.0",
                        "id": 1,
                        "method": "accountSubscribe",
                        "params": [
                            TARGET_WALLET,
                            {"encoding": "jsonParsed", "commitment": "confirmed"}
                        ]
                    })
                log.info(f"🐼 WebSocket connected, listening...")

                async for msg in ws:
                    if msg.type == aiohttp.WSMsgType.TEXT:
                        data = json.loads(msg.data)
                        # Account changed — fetch latest tx
                        if "params" in data:
                            txs = await get_wallet_transactions(session, TARG
                            for tx in txs:
                                sig = tx.get("signature", "")
                                if sig in seen_txs:
                                    continue
                                seen_txs.add(sig)

```

```

log.info(f"📧 New tx from target: {sig[:16]}).

transfers_out, token_buys = parse_transaction

# Copy direct token buys
for mint in token_buys:
    log.info(f"🎯 Target bought {mint[:8]}! C
    await buy_token(session, mint, reason="Di

# Follow transfers to new wallets
for t in transfers_out:
    new_wallet = t["to"]
    if (new_wallet not in followed_wallets an
        len(followed_wallets) < MAX_FOLLOWED
        new_wallet != TARGET_WALLET):
    log.info(f"💰 Transfer {t['sol']:.3f}
    followed_wallets[new_wallet] = True
    asyncio.create_task(watch_new_wallet(

except Exception as e:
    log.error(f"WebSocket error: {e} - reconnecting in 5s...")
    await asyncio.sleep(5)

async def position_monitor(session):
    """Monitor open positions and sell on 2x or -20%."""
    while True:
        for mint, pos in list(active_positions.items()):
            price = await get_token_price(session, mint)
            if price == 0:
                continue
            entry = pos["entry_price"]
            if entry == 0:
                continue
            pnl_pct = (price - entry) / entry * 100
            log.info(f"📊 {mint[:8]}... PnL: {pnl_pct:+.1f}%")

            if pnl_pct >= 100:
                await sell_token(session, mint, reason=f"✅ +{pnl_pct:.0f}% TP hi
            elif pnl_pct <= -20:
                await sell_token(session, mint, reason=f"🔴 {pnl_pct:.0f}% SL hit

        await asyncio.sleep(15)

async def main():
    log.info("=" * 50)
    log.info("  WALLET FOLLOWER BOT")
    log.info(f"  Target: {TARGET_WALLET[:8]}...")

```

```
log.info(f"  Trade size: {TRADE_SOL} SOL")
log.info(f"  TP: +100% | SL: -20%")
log.info(f"=" * 50)

if not keypair:
    log.error("Cannot start without valid PRIVATE_KEY!")
    return

async with aiohttp.ClientSession() as session:
    await asyncio.gather(
        monitor_target(session),
        position_monitor(session),
    )

if __name__ == "__main__":
    asyncio.run(main())
```